Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026

Atividade Prática – Aula 04

Tema – Qualidade de Código com ESLint e Prettier no CI.

Entrega da Atividade

1. Exporte/salve o arquivo de acordo com o padrão: **IEC-Atividade-Aula04.pdf**
2. Enviar a Atividade em PDF no GitHub:
    1. Acesse seu repositório da disciplina: **FATEC-JCR-4DSM-IEC-2026-1-seunome**
    2. Se ainda não existir, você deverá criar a pasta da atividade: **P1-Conteudos/Atividades/Atividade-Aula04/README.md**
    3. Faça o **commit** do pdf da atividade feita na pasta.
    4. Em seguida copie o **link** do pdf.
    5. Acesse o **Kanban-IEC-2026-1** da disciplina que se encontra no repositório da professora.
    6. Clique no card: **IEC-Atividade-Aula04**.
    7. No comentário, cole o **link** do **pdf** da atividade que você fez.
    8. Volte ao seu repositório e acesse o seu **Kanban-IEC-2026-1.**
    9. Mova o card **IEC-Atividade-Aula04** para a coluna **Entregue**.
3. Ou, no caso de configurações feitas no repositório do GitHub, enviar o link.

Lista de Exercícios Práticos (continuação da Aula 3)

Exercício 1 – Instalar ESLint e Prettier

Explicação (porquê):

Precisamos instalar essas ferramentas no projeto para padronizar o código desde já.

Passo a passo (como fazer):

1. No terminal, na raiz do projeto:

npm init -y # cria package.json se não existir

npm install eslint prettier --save-dev

- *--save-dev* -> indica que são dependências de desenvolvimento.

2. Verifique se apareceram em *devDependencies* no *package.json*.

Exercício 2 – Configurar ESLint

Explicação (porquê):

O ESLint precisa de um arquivo de configuração.

Passo a passo (como fazer):

1. Rode o comando:

npx eslint –init

1. Responda às perguntas (sugestão):
    - Como você gostaria de usar o ESLint? -> **Para verificar problemas de sintaxe e encontrar problemas**.
    - Qual framework? -> **React**.
    - Usa TypeScript? -> **Não** (se não estiver usando).
    - Formato do arquivo de config -> **JavaScript**.
2. Será criado o arquivo *.eslintrc.js*.

Exercício 3 – Configurar Prettier

Explicação (porquê):

O Prettier precisa de um arquivo para definir estilo.

Passo a passo (como fazer):

1. Na raiz do projeto, crie o arquivo *.prettierrc* com o conteúdo:

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 2
}
```

2. Isso significa:
    1. Sempre usar ; no final.
    2. Usar aspas simples.
    3. Indentação de 2 espaços.

Exercício 4 – Testar ESLint localmente

Explicação (porquê):

Antes de integrar no CI, precisamos garantir que o ESLint funciona na máquina local.

Passo a passo (como fazer):

1. Edite o *index.js* e coloque:

```
const x= 10
console.log(x)
```

2. Rode o comando:

```
npx eslint .
```

3. O ESLint deve apontar problemas (espaços, ponto e vírgula, etc.).
4. Rode o Prettier para corrigir automaticamente:

```
npx prettier --write .
```

Exercício 5 – Integrar ESLint no GitHub Actions

Explicação (porquê):

Agora o pipeline vai rodar o ESLint automaticamente.

Passo a passo (como fazer):

1. Abra o arquivo *.github/workflows/ci.yml*.
2. Substitua o conteúdo por:

```
name: CI com ESLint

on:
  push:
    branches: [ dev ]
  pull_request:
    branches: [ dev ]

jobs:
  lint:
    runs-on: ubuntu-latest

    steps:
      # 1. Baixar o código
      - name: Checkout do repositório
        uses: actions/checkout@v4
```

```
# 2. Instalar Node.js
- name: Configurar Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '18'

# 3. Instalar dependências
- name: Instalar dependências
  run: npm install

# 4. Rodar ESLint
- name: Rodar ESLint
  run: npx eslint .
```

3. Faça commit na *dev*.
4. Vá em **Actions** -> veja o pipeline rodando.
5. Se houver erro de estilo, o pipeline falhará.